

Тема 9. Конфигурация проекта и логика событий



хекслет колледж



Цель занятия:

Научить студентов «упаковывать» проект профессионально и понимать событийную природу Node.js.

Учебные вопросы:

1. Безопасность и гибкость: dotenv и process.env
2. Логика событий: Модуль events

1. Безопасность и гибкость: `dotenv` и `process.env`

Код - это общественное достояние (Git), а настройки - это личная тайна.

В профессиональной разработке нельзя записывать настройки (пароли, ключи API, порты) прямо в исходный код.

Если такой код попадет в публичный репозиторий, проект будет скомпрометирован.

Глобальный объект `process.env`

В Node.js есть встроенный объект `process.env`. В нем хранятся переменные окружения - настройки той операционной системы, в которой запущен ваш скрипт.

Например, `process.env.USER` вернет имя пользователя в системе.

Мы можем использовать этот объект для передачи настроек в наше приложение.

Зачем нужен модуль `dotenv`?

Настраивать переменные в Windows или Linux вручную перед каждым запуском неудобно.

Модуль `dotenv` позволяет хранить все настройки в простом текстовом файле, который автоматически «подгружается» в `process.env` при старте программы.

Практическая реализация

Шаг 1: Установка

Так как эта библиотека нужна для работы самого приложения, устанавливаем её как обычную зависимость:

```
PS C:\Users\user\Documents\nodeproject\demo-project> npm install dotenv  
added 1 package, and audited 36 packages in 2s
```

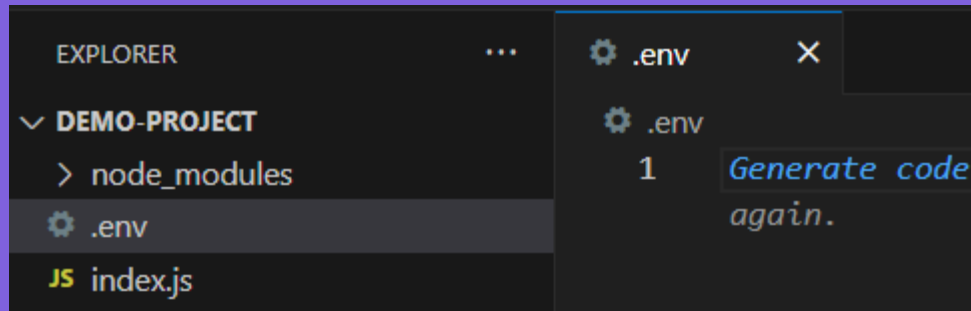

Шаг 2: Создание файла .env

В корне проекта создается файл с названием .env (точка в начале обязательна).

В левой панели (Explorer) нажмите правой кнопкой мыши на пустом месте (или нажмите иконку листочка с плюсиком).

Введите имя файла: **.env**

Нажмите Enter.



В нем пишутся пары ключ-значение без пробелов:

⚙ .env

```
1  PORT=3000
2  API_KEY=my_secret_key_123
3  DB_URL=mongodb://localhost:27017/users_db
4
```

Шаг 3: Использование в коде

Чтобы Node.js «увидел» эти данные, библиотеку нужно подключить в самом начале главного файла (index.js):

```
require('dotenv').config();

// Теперь данные доступны в process.env
const port = process.env.PORT || 5000;
const apiKey = process.env.API_KEY;

console.log(`Сервер будет запущен на порту: ${port}`);
```

Золотое правило безопасности: `.gitignore`

Файл `.env` никогда не должен попадать в Git (на GitHub/GitLab).

Создайте файл `.gitignore`.

Добавьте в него строку: `.env`.

Чтобы другие разработчики знали, какие настройки нужны проекту, принято создавать файл-шаблон `.env.example`, где ключи есть, а реальных секретных значений (паролей) - нет.

Итог:

`process.env` - встроенный объект Node.js для доступа к настройкам окружения.

`.env` - файл-хранилище секретов и настроек.

`dotenv` - мост, который переносит данные из файла в объект `process.env`.

`.gitignore` - обязательный щит, защищающий ваш `.env` от публикации.

2. Логика событий: Модуль events

Большинство встроенных модулей Node.js (включая будущий http, fs или stream) построены на базе событийной архитектуры.

Это означает, что объекты периодически испускают события, а другие объекты на них «подписываются».

Концепция: Издатель и Слушатель

Представьте кнопку в веб-интерфейсе.

Кнопка сама по себе ничего не делает. Но мы можем сказать: «Когда по тебе кликнут - выполни эту функцию».

В Node.js это работает так же, только вместо кликов могут быть любые события: приход данных, завершение загрузки или ошибка.

Класс EventEmitter

Основной инструмент для работы с событиями - класс EventEmitter.

Шаг 1: Подключение и создание

```
const EventEmitter = require('events');  
  
const myEmitter = new EventEmitter();
```


Шаг 2: Регистрация слушателя (.on)

Мы говорим программе, что делать, когда событие произойдет.

```
myEmitter.on('musicStart', (trackName) => {  
  console.log(`Играет трек: ${trackName}`);  
});
```

Шаг 3: Генерация события (.emit)

Мы «кричим» о том, что событие случилось.

```
myEmitter.emit('musicStart', 'The Beatles - Yesterday');
```

Ключевые методы, которые нужно знать:

`on(eventName, listener)` - подписывается на событие.
Будет срабатывать каждый раз, когда оно происходит.

`once(eventName, listener)` - подписывается на событие,
но срабатывает только один раз, после чего подписка
удаляется.

`emit(eventName, [args])` - вызывает событие и передает
в него данные.

Почему это важно для понимания HTTP?

Когда мы перейдем к теме сервера, станет понятно, что сервер в Node.js - это тоже **EventEmitter**.

Нам не нужно писать бесконечный цикл **while(true)**, который проверяет почту или запросы.

Мы просто напомним: **server.on('request', (req, res) => { ... })**.

Node.js - это реактивная среда. Программа не идет по строгому списку команд сверху вниз, а «живет» и ждет сигналов (событий) от системы, пользователя или сети.

Практика

Создадим сервис, который следит за состоянием сервера.

Если температура процессора или нагрузка превышает норму, сервис должен «прокричать» об этом.

Подготовка окружения (5 минут)

Создаем папку и инициализируем проект,
в терминале:

```
mkdir node-practice && cd node-practice  
npm init -y
```

Установлюємо залежності:

`npm i dotenv`

`npm i -D nodemon`

Создаем файл .env с содержимым:

PORT=3000

SAFE_TEMP=70

Напишем код в `index.js`

Этот код наглядно показывает, как события отделяют логику «наблюдения» от логики «реакции».

```
require("dotenv").config();  
const EventEmitter = require("events");  
  
// 1. Инициализация настроек  
const PORT = process.env.PORT || 5000;  
const MAX_TEMP = process.env.SAFE_TEMP || 60;  
  
// 2. Создаем монитор событий  
const monitor = new EventEmitter();
```

```
// 3. Подписываемся на события (Реакция)
monitor.on("overheat", (currentTemp) => {
  console.log(
    `[ВНИМАНИЕ]: Перегрев! Температура ${currentTemp}°C превышает норму в ${MAX_TEMP}°C`,
  );
});

monitor.on("status", (msg) => {
  console.log(`[LOG]: ${msg}`);
});
```

```
// 4. Имитация работы системы (Издатель событий)
console.log(`Система мониторинга запущена на порту ${PORT}...`);

setInterval(() => {
  // Генерируем случайную температуру от 40 до 90
  const temp = Math.floor(Math.random() * 50) + 40;

  if (temp > MAX_TEMP) {
    monitor.emit("overheat", temp);
  } else {
    monitor.emit("status", `Температура в норме: ${temp}°C`);
  }
}, 2000);
```

Настройка `package.json`

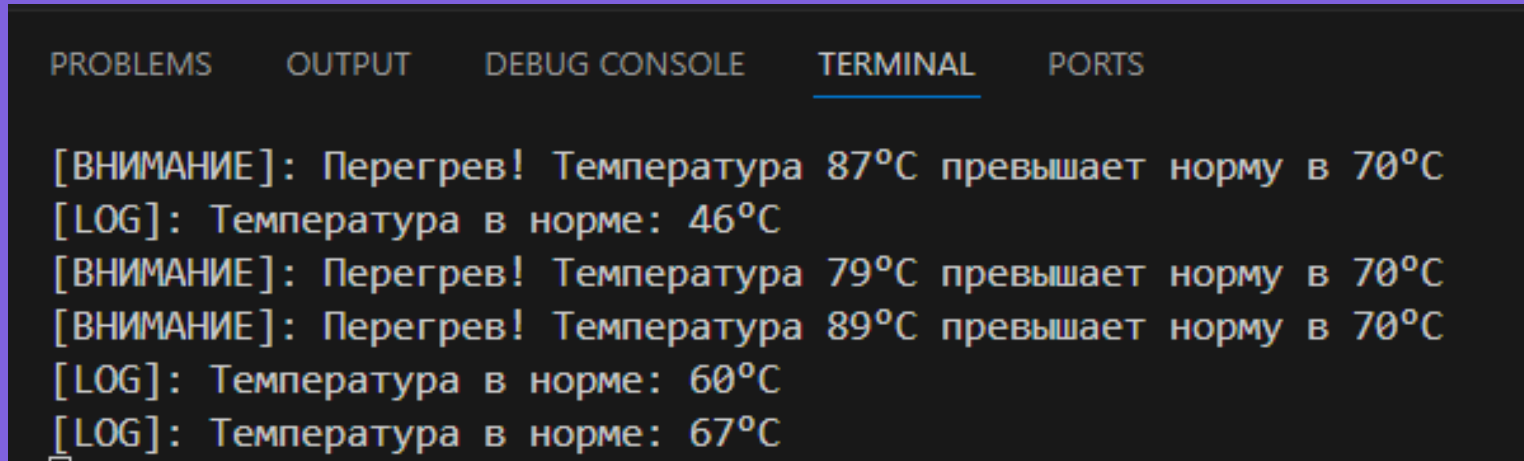
Прописываем скрипты, чтобы запуск стал удобным:

```
{  
  "name": "package.json",  
  "version": "1.0.0",  
  "main": "index.js",  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```

Запуск: Выполните:

`npm run dev`

Убедитесь, что логи пошли в консоль каждые 2 секунды.



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays a series of log messages in Russian, alternating between warning messages about temperature and log messages indicating the temperature is within the norm. The messages are as follows:

```
[ВНИМАНИЕ]: Перегрев! Температура 87°C превышает норму в 70°C  
[LOG]: Температура в норме: 46°C  
[ВНИМАНИЕ]: Перегрев! Температура 79°C превышает норму в 70°C  
[ВНИМАНИЕ]: Перегрев! Температура 89°C превышает норму в 70°C  
[LOG]: Температура в норме: 60°C  
[LOG]: Температура в норме: 67°C
```

Выводы:

- **Разделение ответственности:** Настройки проекта (порты, ключи, пароли) выносятся в файл `.env`, а код остается универсальным. Это стандарт безопасности.
- **Глобальный доступ:** Модуль `dotenv` загружает данные из файла в объект `process.env`, откуда их может прочитать любой скрипт в приложении.
- **Автоматизация рутины:** Секция `scripts` в `package.json` позволяет создавать короткие команды для запуска (`npm start`, `npm run dev`), скрывая сложные параметры терминала.
- **Событийная архитектура:** Node.js работает по принципу «Реакция на событие». Модуль `events` позволяет объектам общаться друг с другом, не зная деталей реализации соседа.
- **Разделение зависимостей:** Инструменты для программиста (например, `nodemon`) ставятся в `devDependencies`, а библиотеки для работы приложения - в основные `dependencies`.

Контрольные вопросы

Почему нельзя добавлять файл `.env` в Git-репозиторий?

В чем разница между командами `npm start` и `npm run dev`?

Зачем нужен флаг `-D` (или `--save-dev`) при установке пакетов?

Что делает метод `.emit('eventName', data)`?

Как заставить `nodemon` перезагружать проект не только при изменении JS-файлов, но и при правке настроек в `.env`?